UNIVERSITÉ SORBONNE PARIS NORD

Institut Galilée

Rapport de Programmation Fonctionnelle

Utilisation d'OCaml pour illustrer des analogies de tautologie dans les formules propositionnelles

Formation : Double licence Mathématiques Informatique
Unité d'enseignement : Programmation Fonctionnelle
Responsable de TD : Christian CODOGNET

Mathura SANTHALINGAM :
Youssef EL OTMANI :

Sommaire

I) Présentation du problème 2

II) Solution algorithmique et listing commenté 2

Étape 1 : Transformation d'une formule F de type propositionnelle en une formule G de type conditionnelle équivalente en définissant la fonction trad 3

Etape 2 : Mise en forme normale de la formule G en une nouvelle formule H de type cond équivalente en définissant for_nor 5

Étape 3 : Détermination du caractère tautologique (ou non) d'une formule h 6

Étape 3a : Définition de la fonction appartient 6

Étape 3b : Définition de la fonction eval 7

III) Jeux d'essais 8

Exemple de l'étape 1 8

Exemple de l'étape 2 9

Exemple de l'étape 3 10

Les classiques de la Tautologie 11

Difficultés rencontrées 12

IV) Conclusion 13

I) Présentation du problème

Ce rapport explore l'implémentation d'un algorithme en OCaml visant à déterminer si une formule propositionnelle est une tautologie en s'appuyant sur deux types distincts : `prop` pour les formules propositionnelles et `cond` pour les formules conditionnelles.

Une tautologie est un aspect primordial de la logique propositionnelle. Il s'agit d'une formule propositionnelle qui est vraie pour toutes les distributions possibles de valeurs de vérité de ses variables. Cela nous amène donc à nous demander *comment vérifier si une formule propositionnelle est une tautologie.*

L'objectif principal est de simplifier les propositions logiques complexes et de les traduire en formules conditionnelles afin de faciliter leur évaluation.

II) Solution algorithmique et listing commenté

Pour traiter et résoudre de manière algorithmique le problème de la vérification de la tautologie pour des formules propositionnelles en utilisant les types prop et cond en OCaml, nous pouvons concevoir une solution en suivant les étapes de transformation, de mise en forme normale et d'évaluation.

```
type prop =
| V of int (* variable d'entier *)
| Vrai (* constante vrai *)
| Faux (* constante faux *)
| Non of prop (* négation d'une proposition *)
| Et of prop * prop (* conjonction de deux propositions *)
| Ou of prop * prop (* disjonction de deux propositions *)
| Imp of prop * prop (* implication entre deux propositions *)
| Equiv of prop * prop (* équivalence entre deux propositions *)
```

On définit le type `prop` qui représente le type des valeurs que peuvent prendre une formule propositionnelle :

- variable de type entier qui peut être associée à une constante `V(i)`.
- constantes de vérité : `Vrai` et `Faux`.
- connecteurs logiques : `Et`, `Ou` (inclusif), l'implication `Imp` et l'équivalence `Equiv`.

```
type cond =
| W of int (* variable dans le contexte des conditions *)
| Vrai_bis (* constante vrai dans une formule conditionnelle *)
| Faux_bis (* constante faux dans une formule conditionnelle *)
| Si of cond * cond * cond (* conditionnelle avec une condition et
deux branches *)
```

Le type `cond` quant à lui, à :

- variables de type entier `W(i)`
- constantes de vérités : `Vrai_bis` et `Faux_bis`,
- l'unique connecteur `Si`.

Pour répondre à ce problème, notre solution algorithmique se découpe en trois étapes :

Étape 1 : Transformation d'une formule F de type propositionnelle en une formule G de type conditionnelle équivalente en définissant la fonction `trad`

```
(*Etape 1*)
let rec trad fp = match fp with
    V(i) -> W(i)(* traduction de la variable prop en variable cond *)
  |Vrai -> Vrai_bis (* traduction de vrai prop en vrai cond *)
  |Faux -> Faux_bis (* traduction de faux prop en faux cond *)
  |Non (prop) -> Si ((trad prop), Faux_bis, Vrai_bis)
(* traduction la négation d'une formule prop en conditionnelle *)
  |Et (prop1, prop2) -> Si ((trad prop1), (trad prop2), Faux_bis)
(* traduction de la conjonction de deux propositions prop en
conditionnelle *)
  |Ou (prop1, prop2) -> Si ((trad prop1), Vrai_bis, (trad prop2))
(* traduction de disjonction entre deux propositions prop en
conditionnelle *)
  |Imp (prop1, prop2) -> trad (Ou(Non (prop1), prop2))
(* traduction de l'implication entre deux propositions prop en
conditionnelle *)
  |Equiv (prop1, prop2) -> trad (Et ( Imp(prop1, prop2), Imp(prop2,
prop1) ) );;
(* traduction de l'équivalence entre deux propositions prop en
conditionnelle *)
```

Tout d'abord, on pose l'équivalent des valeurs de vérité `Vrai` et `Faux` comme étant `Vrai_bis` et `Faux_bis`, de même, chaque variable `V(i)` devient `W(i)`, avec i un entier.
Afin d'écrire toute formule propositionnelle en une formule conditionnelle, il est aussi primordial de construire les équivalents de tous les constructeurs en des formules ne contenant que le connecteur "Si". Ainsi, nous les avons traduites comme suit :

Remarque : la fonction `Si(a,b,c)` *équivaut à dire "si a vrai, on renvoie b sinon c"*

Tout au long des étapes, nous utiliserons une correspondance de motifs (pattern-matching) avec les mots clés `match x with`. Cela permet de comparer la valeur de `x` avec plusieurs motifs et d'exécuter du code en fonction du motif correspondant.
Nous utiliserons notamment `let a=b in` qui permet de lier une valeur `b` à un nom `a` dans un contexte local spécifique, délimité par `in`.

Pour mimiquer le comportement de `Non` d'une formule : on traduit la formule propositionnelle passée en argument en une formule conditionnelle, en appelant la fonction `trad`**.** On examine sa valeur de vérité **pour renvoyer la valeur** `Faux_bis` **si l'argument est** `Vrai`**, et vice versa.**

Pour exprimer `Et` de deux formules sous forme conditionnelle : une fois de plus, on traduit les formules propositionnelles passées en argument en des formules conditionnelles par un appel à `trad`**.** Cette fois-ci, on examine leurs valeurs de vérité **pour renvoyer** `Faux_bis` **si le premier argument (i.e la première formule) est** `Faux`, **et renvoyer la valeur de vérité de la deuxième formule dans le cas échéant.**

En ce qui concerne le `Ou` de deux formules : à nouveau, on commence par traduire les formules propositionnelles passées en argument en des formules conditionnelles, avec `trad`, et on examine leurs valeurs de vérité. On doit d'abord évaluer la valeur de vérité du premier argument, **si celui-ci est** `Vrai`, **on renvoie la valeur** `Vrai_bis`**, dans le cas échéant, on renvoie la valeur de vérité du deuxième.**

Une fois que nous avons traduit le `Ou` et le `Et`, nous pouvons les utiliser pour exprimer les implications et équivalences. En effet, puisque "A implique B" équivaut à "non A ou B", nous pouvons tout simplement **traduire** `Imp` **de deux formules par** `trad (Non` **de la première formule** `Ou` **la deuxième).**

Enfin, en ce qui concerne `Equiv`, qui est une double implication, il suffit de **traduire l'implication dans un sens** `Et` **dans l'autre** : `trad (Imp` **de première formule à la deuxième** `Et` **de la deuxième à la première**) en appliquant les traductions faites plus tôt.

Une fois que nous avons traduit les différents constructeurs des formules propositionnelles de manière à avoir des formules conditionnelles, nous devons rendre ces dernières normales.

Etape 2 : Mise en forme normale de la formule G en une nouvelle formule H de type `cond` équivalente en définissant `for_nor`

```
(*Etape 2*)
let rec for_nor fc = match fc with
    W(i)-> W(i) (* Si fc est de la forme W(i), retourner W(i) *)
  |Vrai_bis -> Vrai_bis (* Si fc est Vrai_bis, retourner Vrai_bis *)
  |Faux_bis -> Faux_bis (* Si fc est Faux_bis, retourner Faux_bis *)
  |Si(j,d,e) -> match j with
(* Si fc est de la forme Si(j,d,e), alors vérifier le motif de j *)
      W(i)-> Si(j, (for_nor d), (for_nor e))
(* Si j est W(i), alors retourner Si(j, (for_nor d), (for_nor e)) *)
    |Vrai_bis -> Si (j, for_nor d, for_nor e)
(* Si j est Vrai_bis, alors retourner Si(j, for_nor d, for_nor e) *)
    |Faux_bis -> Si (j, for_nor d, for_nor e)
(* Si j est Faux_bis, alors retourner Si(j, for_nor d, for_nor e) *)
    |Si(a,b,c) -> for_nor (Si ( a, Si(b, d, e), Si(c, d, e)) );;
(* Si j est Si(a,b,c), alors appeler récursivement for_nor avec une
nouvelle structure Si *)
```

Pour cela, en supposant la formule préalablement rendue conditionnelle, il nous suffit de nous débarrasser de toutes les occurrences de la forme `Si(Si(a,b,c),d,e)`. En effet, cela permettra de simplifier davantage la formule propositionnelle et donc de rendre son évaluation (si c'est une tautologie ou non) plus directe.

Pour ce faire, tout d'abord, si la formule ne contient pas de `Si`, on la renvoie telle quelle : cela comprend les cas des constantes `Vrai_bis`, `Faux_bis` et des variables `W(i)`, avec i un entier. Dans le cas où la formule est bien sous forme de condition `(Si)`, on vérifie la forme du **premier argument** :

- Si ce dernier est sous la forme d'un `Si`, puisque **(1)** `Si(Si(a,b,c),d,e)` équivaut à **(2)** `Si(a,Si(b,d,e),Si(c,d,e))` on réécrit notre formule selon le modèle **(2)** puis on applique `for_nor` à cette dernière.

- Si ce dernier n'est pas sous forme de `Si`, on renvoie la formule principale avec le premier argument inchangé tout en appliquant la fonction `for_nor` sur les deux autres arguments.

Ces deux étapes sont nécessaires afin d'évaluer la formule et savoir s'il s'agit d'une tautologie ou non.

Étape 3 : Détermination du caractère tautologique (ou non) d'une formule `h`

Étape 3a : Définition de la fonction `appartient`

```
(*Etape 3a*)

let rec appartient i e = match e with
    [] -> (false,false)
(* Si la liste e est vide, retourner un couple (false, false) *)

  |(j,d)::ee -> if i = j then (true,d) else (appartient i ee);;
(* Si e est de la forme (j,d)::ee, vérifier si i est égal à j *)
```

Cette fonction prend en entrée un entier `i` et un environnement `e` qui correspond à une liste de couples `(j,b)` avec `j` un entier et `b` un booléen. Le but étant de renvoyer la valeur du booléen associé à `i` si ce dernier appartient à `e`. On distingue différents cas de figures :

- si la liste est vide cela signifie que `i` n'appartient pas à `e` et donc la fonction renvoie le couple `(false, false)`.

- si `i` appartient au premier couple de `e`, la fonction renvoie `(true,` le booléen associé à `i)`.

- si `i` n'appartient pas au premier couple de `e`, on applique récursivement la fonction `appartient` sur la queue de la liste de couples (`e` privé du premier couple).

Étape 3b : Définition de la fonction `eval`

```
(*Etape 3b*)

let rec eval h e = match h with
    Vrai_bis -> true (* Si h est Vrai_bis, retourner true *)
  |Faux_bis -> false (* Si h est Faux_bis, retourner false *)
  |W(i) -> let (a,b) = (appartient i e) in b
(* Si h est de la forme W(i), vérifier l'appartenance de i dans
l'environnement e *)
  |Si(g,f,k) -> match g with
(* Si h est de la forme Si(g,f,k), vérifier la valeur de g *)
      Vrai_bis -> (eval f e)
   (* Si g est Vrai_bis, évaluer f dans l'environnement e *)
    |Faux_bis -> (eval k e)
   (* Si g est Faux_bis, évaluer k dans l'environnement e *)
    |W(i) -> let (a,b) = (appartient i e) in if a
   (* Si g est de la forme W(i), vérifier si i est dans e *)
        then if b then (eval f e) else (eval k e)
   (* Si i est dans e : vérifier la valeur de i dans e
       puis selon cette valeur, évaluer f ou k dans e *)
        else let e1 = (i, true)::e and e2 = (i, false)::e in
 ((eval f e1)&&(eval k e2))
   (* Si i n'est pas dans e, étendre e avec i évalué à vrai et faux *)
    |Si(a,b,c) -> false ;;
    (* Si h est de la forme Si(a,b,c), retourner faux *)
```

La fonction `eval` prend en entrée une formule logique `h` et un environnement `e` (une liste de couple d'entiers et de booléens). Elle évalue la formule `h` en fonction des valeurs associées dans `e` et retourne un booléen. En pratique, si la formule `h` est :

- `Vrai_bis`, la fonction retourne `true`.
- `Faux_bis`, la fonction retourne `false`.
- de forme `W(i)`, la fonction `appartient` cherche le booléen associé à `i` dans `e`, et le renvoie s' il le trouve, sinon renvoie `false` (si le couple n'est pas dans `e`).
- de forme `Si` le 1er argument est :
  - `Vrai_bis`, la fonction évalue le 2ème avec le même environnement `e`.
  - `Faux_bis`, la fonction évalue le 3ème avec le même environnement `e`.
  - de la forme `W(i)`, la fonction cherche la valeur de `i` dans l'environnement `e` :
    - Si `i` est trouvé et associé à `true`, on évalue le 2ème argument avec le même e.
    - Sinon, si `i` est trouvé et associé à `false`, la fonction évalue le 3ème argument avec e.
    - Si `i` n'est pas trouvé, la fonction évalue récursivement :
      - le **2ème** argument dans `e1` (i.e l'environnement `e` mais en associant à `i` à `true`.
      - le **3ème** argument dans `e2` (i.e l'environnement `e` mais en associant à `i` à `false`).
      - Puis elle combine les résultats avec l'opérateur `&&`.

Remarque : si la formule f est un `Si`*, (ce qui ne devrait pas arriver puisque la formule passée en argument de* `eval` *est sous forme normale),* `eval` *renvoie* `false`*.*

III) Jeux d'essais

Exemple de l'étape 1

Non :

```
#trad (Non (Vrai)) ;;
- : cond = Si (Vrai_bis, Faux_bis, Vrai_bis)
```

Et :

```
#trad ET(Vrai, Faux);;
- : cond = Si (Vrai_bis, Faux_bis, Faux_bis)
```

Ou :

```
#trad (Ou(V(6), Faux )) ;;
- : cond = Si (W 6, Vrai_bis, Faux_bis)
```

Imp :

```
#trad (Imp (Vrai, Vrai)) ;;
- : cond = Si (Si (Vrai_bis, Faux_bis, Vrai_bis), Vrai_bis, Vrai_bis)
```

car : `Imp (Vrai, Vrai) = trad (Ou(Non (Vrai), Vrai)) = Si ((trad Non(Vrai) ), Vrai_bis, (trad Vrai)) = Si ((Si ((trad Vrai ), Faux_bis, Vrai_bis) ), Vrai_bis, Vrai_bis)`

Equiv :

```
#trad (Equiv(Vrai, V(5) ) );;
- : cond =
Si (Si (Si (Vrai_bis, Faux_bis, Vrai_bis), Vrai_bis, W 5),
 Si (Si (W 5, Faux_bis, Vrai_bis), Vrai_bis, Vrai_bis), Faux_bis)
```

car : `trad(Equiv(Vrai,V(5)))=trad(Et(Imp(Vrai,V(5)),Imp(V(5),Vrai)) )`
`= Si ((trad Imp(Vrai, V(5))), (trad Imp(V(5), Vrai)), Faux_bis)`
`= Si(trad(Ou(Non(Vrai),V(5)))),(trad(Ou(Non(V(5)),Vrai))), Faux_bis)`
`= Si (Si ((trad Non (Vrai)), Vrai_bis, (trad V(5))), Si ((trad Non (V(5))), Vrai_bis, (trad Vrai)), Faux_bis)`
`= Si (Si (Si ( Vrai_bis, Faux_bis, Vrai_bis), Vrai_bis, W(5)), Si ( Si ( W(5), Faux_bis, Vrai_bis), Vrai_bis, Vrai_bis), Faux_bis)`

Et (Non) :

```
#trad (Et(Vrai, Non( V(8)))) ;;
- : cond = Si (Vrai_bis, Si (W 8, Faux_bis, Vrai_bis), Faux_bis)
```

Non (Ou) :

```
#trad (Non ( Ou ( Vrai , V(6) ) ) ) ;;
- : cond = Si (Si (Vrai_bis, Vrai_bis, W 6), Faux_bis, Vrai_bis)
```

Ou (Imp) :

```
#trad (Ou( Imp (Vrai, Faux) , Vrai )) ;;
- : cond =
Si (Si (Si (Vrai_bis, Faux_bis, Vrai_bis), Vrai_bis, Faux_bis),
Vrai_bis, Vrai_bis)
```

Exemple de l'étape 2

Sans Si (Si (a, b, c), d, e) :

```
#for_nor (Si(Vrai_bis, Si (W 8, Faux_bis, Vrai_bis), Faux_bis)) ;;
- : cond = Si (Vrai_bis, Si (W 8, Faux_bis, Vrai_bis), Faux_bis)
```

Si (Si (a, b , c), d, e) :

```
#for_nor (Si(Si (Vrai_bis, Vrai_bis, W 6), Faux_bis, Vrai_bis)) ;;
- : cond =
Si (Vrai_bis, Si (Vrai_bis, Faux_bis, Vrai_bis),
 Si (W 6, Faux_bis, Vrai_bis))
```

Si (a, Si (Si (b, c, d), e, f), g) :

```
#for_nor (Si ( Vrai_bis, Si( Si( Vrai_bis , Vrai_bis , Faux_bis ) ,
Vrai_bis, Faux_bis ) , Vrai_bis ) );;
- : cond =
Si (Vrai_bis,
 Si (Vrai_bis, Si (Vrai_bis, Vrai_bis, Faux_bis),
  Si (Faux_bis, Vrai_bis, Faux_bis)), Vrai_bis)
```

car :

```
for_nor (Si ( Vrai_bis, Si( Si( Vrai_bis , Vrai_bis , Faux_bis ) ,
Vrai_bis, Faux_bis ) , Vrai_bis ) )
= Si (Vrai_bis, for_nor Si( Si( Vrai_bis , Vrai_bis , Faux_bis ) ,
Vrai_bis, Faux_bis ), for_nor Vrai_bis)
= Si (Vrai_bis, for_nor (Si ( Vrai_bis, Si(Vrai_bis, Vrai_bis,
Faux_bis), Si(Faux_bis, Vrai_bis, Faux_bis)) ), for_nor Vrai_bis)
= Si (Vrai_bis, Si ( Vrai_bis,for_nor Si(Vrai_bis, Vrai_bis,
Faux_bis),for_norSi(Faux_bis, Vrai_bis,Faux_bis))),for_nor Vrai_bis)
= Si (Vrai_bis, Si ( Vrai_bis, Si(Vrai_bis, for_nor Vrai_bis, for_nor
Faux_bis), for_nor Si(Faux_bis, Vrai_bis, Faux_bis) ) ), for_nor
Vrai_bis)
= Si (Vrai_bis, Si ( Vrai_bis, Si(Vrai_bis,Vrai_bis, Faux_bis),
for_nor Si(Faux_bis, Vrai_bis, Faux_bis) ) ), for_nor Vrai_bis)
= Si (Vrai_bis, Si ( Vrai_bis, Si(Vrai_bis,Vrai_bis, Faux_bis),
Si(Faux_bis, for_nor Vrai_bis, for_nor Faux_bis) ) ), for_nor
Vrai_bis)
= Si (Vrai_bis, Si ( Vrai_bis, Si(Vrai_bis,Vrai_bis, Faux_bis),
Si(Faux_bis, Vrai_bis, Faux_bis) ) ), for_nor Vrai_bis)
= Si (Vrai_bis, Si ( Vrai_bis, Si(Vrai_bis,Vrai_bis, Faux_bis),
Si(Faux_bis, Vrai_bis, Faux_bis) ) ), Vrai_bis)
```

Si (a, b, Si ((Si (c, d, e), g, h)) :

```
#for_nor (Si ( Vrai_bis,Vrai_bis ,Si( Si( Vrai_bis , Vrai_bis ,
Faux_bis ) , Vrai_bis, Faux_bis )) );;
- : cond =
Si (Vrai_bis, Vrai_bis,
 Si (Vrai_bis, Si (Vrai_bis, Vrai_bis, Faux_bis),
  Si (Faux_bis, Vrai_bis, Faux_bis)))
```

Si (Si (Si (a, b, c), d, e), f, g) :

```
#for_nor ( Si (Si (Si (Vrai_bis, Faux_bis, Vrai_bis), Vrai_bis,
Faux_bis), Vrai_bis, Vrai_bis)) ;;
- : cond =
Si (Vrai_bis,
 Si (Faux_bis, Si (Vrai_bis, Vrai_bis, Vrai_bis),
  Si (Faux_bis, Vrai_bis, Vrai_bis)),
 Si (Vrai_bis, Si (Vrai_bis, Vrai_bis, Vrai_bis),
  Si (Faux_bis, Vrai_bis, Vrai_bis)))
```

Exemple de l'étape 3

Si (Si (a, b, c), d, e) [] :

```
#eval (Si(Si(Vrai_bis,Vrai_bis, W 6),Faux_bis,Vrai_bis)) [] ;;
- : bool = false
```

Remarque : puisque l'argument de `eval` est censé être une formule conditionnelle normale, nous ne serons jamais confrontés à ce cas, peu importe l'environnement.

Sans W(i) :

a) Tautologie :

```
# eval (Si(Vrai_bis, Vrai_bis, Faux_bis)) [] ;;
- : bool = true
```

b) Pas Tautologie :

```
# eval (Si(Vrai_bis, Faux_bis, Faux_bis)) [] ;;
- : bool = false
```

Avec W(i) :

a) Tautologie :

```
# eval (Si(W(1), Vrai_bis, Si(W(2), Vrai_bis, Vrai_bis))) [] ;;
- : bool = true
```

car : si `W(1)` est Vrai_bis on renvoie directement `Vrai_bis` donc `true`
et si `W(1)` est false, on renvoie `Si(W(2), Vrai_bis, Vrai_bis)`
et que `W(2)` soit vrai ou faux, on renvoie `Vrai_bis` donc `true`

b) Pas Tautologie :

```
# eval (Si(W(1), Vrai_bis, Faux_bis)) [] ;;
- : bool = false
```

car :

- avec `(1,true)` appartient à `e`
  ```
  eval (Si(W(1), Vrai_bis, Faux_bis)) ((1,true)::[]) ;;
  - : bool = true
  ```
- avec `(1,false)` appartient à `e`
  ```
  eval (Si(W(1), Vrai_bis, Faux_bis)) ((1,false)::[]) ;;
  - : bool = false
  ```

Donc ce n'est pas une tautologie car il aurait fallu que tous les cas mènent à `true`.

Les classiques de la Tautologie

Ainsi, pour appliquer nos trois étapes à des exemples concrets et pouvoir vérifier si une formule propositionnelle est une tautologie, il nous suffira d'utiliser la commande `eval (for_nord ( trad  F)))`

Exemple : Commutativité : A Et B <=> B Et A

```
eval (for_nor(trad(Equiv(Et(V(6),V(2)),Et(V(2),V(6)))))) [] ;;
- : bool = true
```

Exemple : Distributivité : (A Ou B) Et C <=> (A Et C) Ou (B Et C)

```
eval (for_nor(trad( Equiv(Et(Ou(V(1),V(2)),V(3)),
Ou(Et(V(1),V(3)),Et(V(2),V(3)) ) )))) [] ;;
- : bool = true
```

Exemple : Associativité : (A Et B) Et C <=> A Et (B Et C)

```
eval (for_nor(trad(Equiv(Et(Et(V(1),V(2)),V(3)), Et(V(1), Et(V(2)
,V(3) ) ) ) ) ) )  [] ;;
- : bool = true
```

Exemple : Transitivité : A Imp B Et B Imp C => A Imp C

```
eval (for_nor(trad( Imp( Et( Imp(V(1),V(2)) , Imp (V(2),V(3))  )
, Imp(V(1),V(3))   ) ))) [] ;;
- : bool = true
```

Exemple : Double négation Non Non A <=> A

```
eval (for_nor(trad(Equiv(Non(Non(V(1))),V(1))))) [];;
- : bool = true
```

Difficultés rencontrées

- Le choix des exemples.

Solution : nous avons opté pour des exemples pertinents illustrant tous les cas de figures sans pour autant combiner trop de connecteurs car cela aurait été illisible.

- Dans la deuxième étape, distinguer les cas entre appliquer un `Si` et le normaliser. Par exemple, pour normaliser `Si(j, d, e)` dans le cas où `j` est `Vrai_bis` :

  Solution : la fonction `for_nor` devra renvoyer **:**
  `Si (j, for_nor d, for_nor e)` et non `for_nord d`.

- Dans la dernière étape, vérifier l'appartenance ou non d'une variable i dans un environnement e, tout en renvoyant la valeur de vérité qui lui est associée.

  Solution : nous avons décidé de créer une nouvelle fonction récursive `appartient` qui renvoie un couple permettant de distinguer ces cas : la première valeur étant appartenance ou non de i dans et la deuxième désignant la valeur de vérité associée.

- Dans la dernière étape, éviter un Warning quand la formule n'est pas normalisée.

  Solution : rajouter dans `eval` une ligne de code
  `|Si(a,b,c) -> false ;;`
  même si elle n'est pas explicitement demandée.

IV) Conclusion

Pour répondre à notre question qui portait sur l'aspect tautologique ou non d'une formule propositionnelle nous sommes parvenus à définir 4 fonctions : `trad`, `for_nor`, `appartient`, `eval`. Chacune représentant une étape primordiale à notre solution. En effet, la traduction et la normalisation sont nécessaires pour aboutir à une formule facilement évaluable. Ainsi, l'évaluation de la formule dans les différentes distributions de valeurs de vérité permet de déterminer si elle est une tautologie. Cette étape est cruciale car elle teste la validité de la formule dans toutes les situations possibles, assurant ainsi qu'elle est vraie quelle que soit la valeur de vérité attribuée à ses variables.

Enfin, pour savoir si une formule propositionnelle `F` est une tautologie, il nous suffit donc d'entrer la commande :

```
eval (for_nord ( trad  F)))
```